

Lab02 Kaleo256333

Kaleo Campos

```
library(readr)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(tibble)
library(ggcal)
library(ggplot2)
```

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada ($\text{ARRIVAL_DELAY} > 10$)?

##Resposta:

Para cada grupo (dia, mês, cia. aérea), as estatísticas suficientes são duas contagens:

- n: número de voos válidos do grupo;
- k: número desses voos com $\text{ARRIVAL_DELAY} > 10$.

Elas são aditivas: somando o n e o k de todos os lotes de um mesmo grupo, o percentual final é k/n . Não é possível usar a média dos percentuais calculados lote a lote, pois lotes com quantidades diferentes de voos do mesmo grupo pesariam igual, distorcendo o resultado.

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):

-Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US; -Remova observações que tenham valores faltantes em campos de interesse; -Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea; -Para cada grupo em `b.`, determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`; -A função deve receber apenas dois argumentos: -input: o conjunto de dados (referente ao lote em questão); -pos: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
getStats <- function(input, pos) {  
  input %>%  
    filter(AIRLINE %in% c("AA", "DL", "UA", "US"), !is.na(ARRIVAL_DELAY)) %>%  
    mutate(atrasado = ARRIVAL_DELAY > 10) %>%  
    group_by(DAY, MONTH, AIRLINE) %>%  
    summarise(  
      n = n(),  
      atrasados = sum(atrasado),  
      .groups = "drop"  
    )  
}
```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

-Configure o tamanho do lote (chunk) para 100 mil registros; -Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima; -Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
tipos_colunas <- cols_only(  
  AIRLINE = "c",  
  MONTH = "i",  
  DAY = "i",  
  ARRIVAL_DELAY = "d"  
)  
  
stats_por_lote <- read_csv_chunked(  
  file = "flights.csv",  
  callback = DataFrameCallback$new(getStats),
```

```

chunk_size = 100000,
col_types = tipos_colunas
)

```

4. Crie uma função chamada `computeStats` que:

-Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea); -Retorne as informações em um tibble contendo apenas as seguintes colunas: -Cia: sigla da companhia aérea; -Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`); -Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo [0,1].

```

computeStats <- function(stats) {
  stats %>%
    group_by(DAY, MONTH, AIRLINE) %>%
    summarise(
      n          = sum(n),
      atrasados  = sum(atrasados),
      .groups    = "drop"
    ) %>%
    mutate(
      Cia      = AIRLINE,
      Data     = as.Date(sprintf("2015-%02d-%02d", MONTH, DAY)),
      Perc     = atrasados / n
    ) %>%
    select(Cia, Data, Perc) %>%
    arrange(Cia, Data)
}

resultado <- computeStats(stats_por_lote)
resultado

```

```

# A tibble: 1,276 x 3
  Cia    Data      Perc
  <chr> <date>    <dbl>
1 AA    2015-01-01 0.373
2 AA    2015-01-02 0.430
3 AA    2015-01-03 0.538
4 AA    2015-01-04 0.602
5 AA    2015-01-05 0.491
6 AA    2015-01-06 0.366
7 AA    2015-01-07 0.355

```

```

8 AA      2015-01-08 0.307
9 AA      2015-01-09 0.342
10 AA     2015-01-10 0.280
# i 1,266 more rows

```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

-Instale e carregue os pacotes ggcal e ggplot2. -Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`. -Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá: -Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`. -Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`. -Retornar para o usuário a base do calendário criada acima. -Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`. -Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Título")`.

```

pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027")

baseCalendario <- function(stats, cia) {
  sub <- filter(stats, Cia == cia)
  ggcal(sub$Data, sub$Perc)
}

cAA <- baseCalendario(resultado, "AA")
cDL <- baseCalendario(resultado, "DL")
cUA <- baseCalendario(resultado, "UA")
cUS <- baseCalendario(resultado, "US")

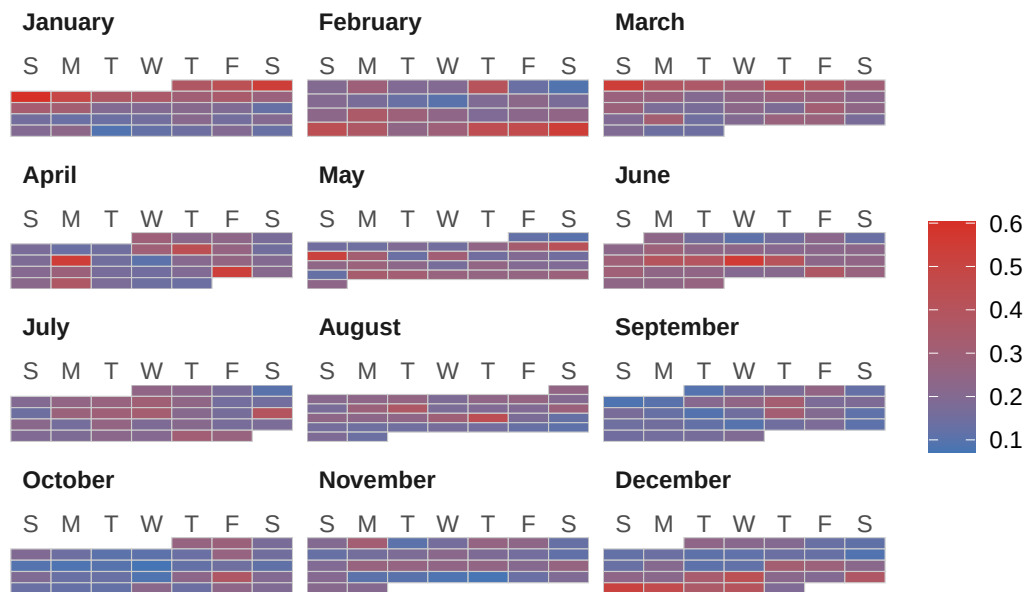
```

```

cAA + pal + ggtitle("American Airlines (AA)")

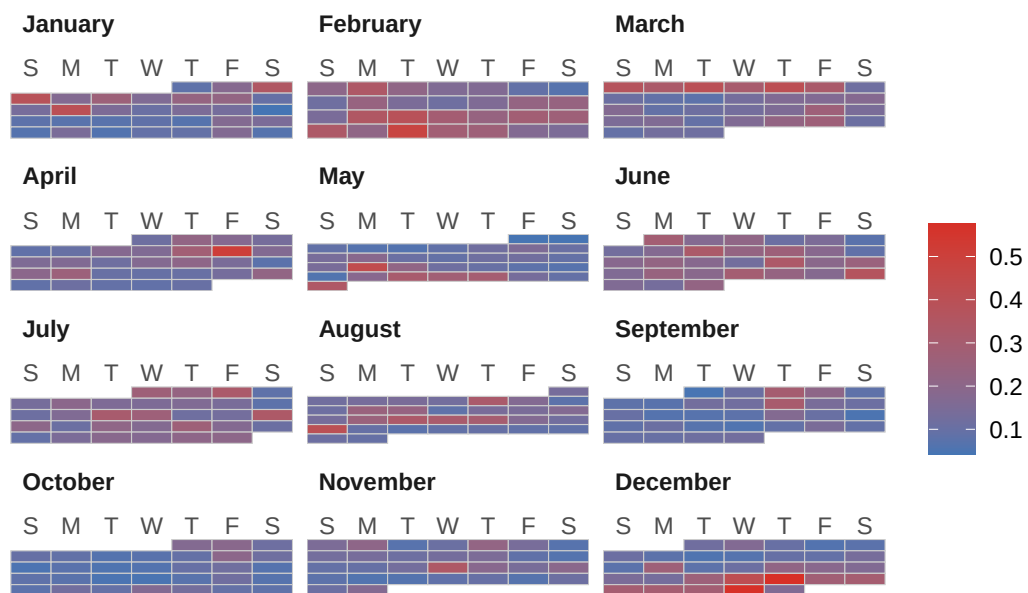
```

American Airlines (AA)



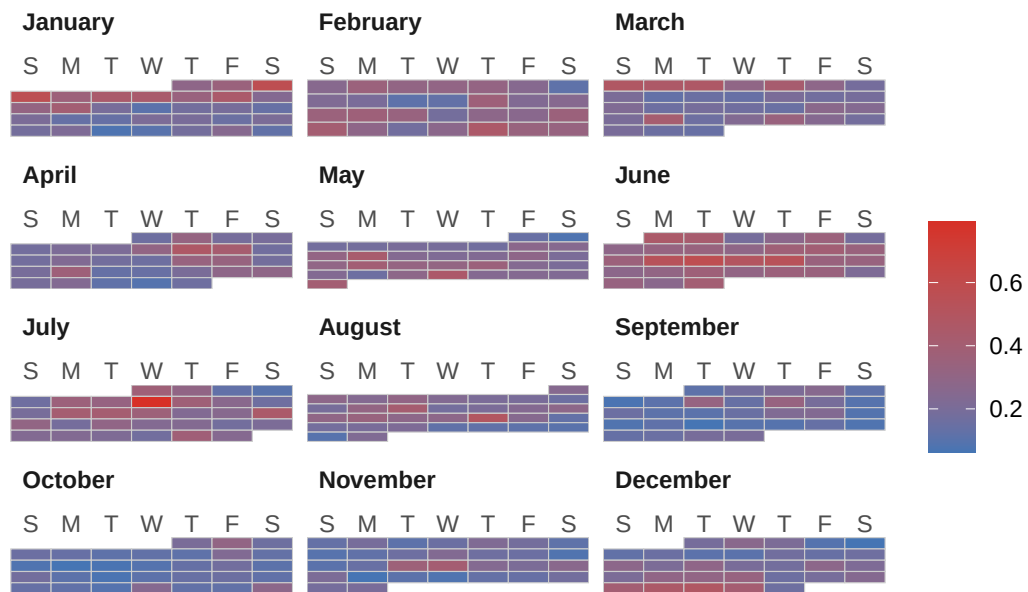
```
cDL + pal + ggtitle("Delta Air Lines (DL)")
```

Delta Air Lines (DL)



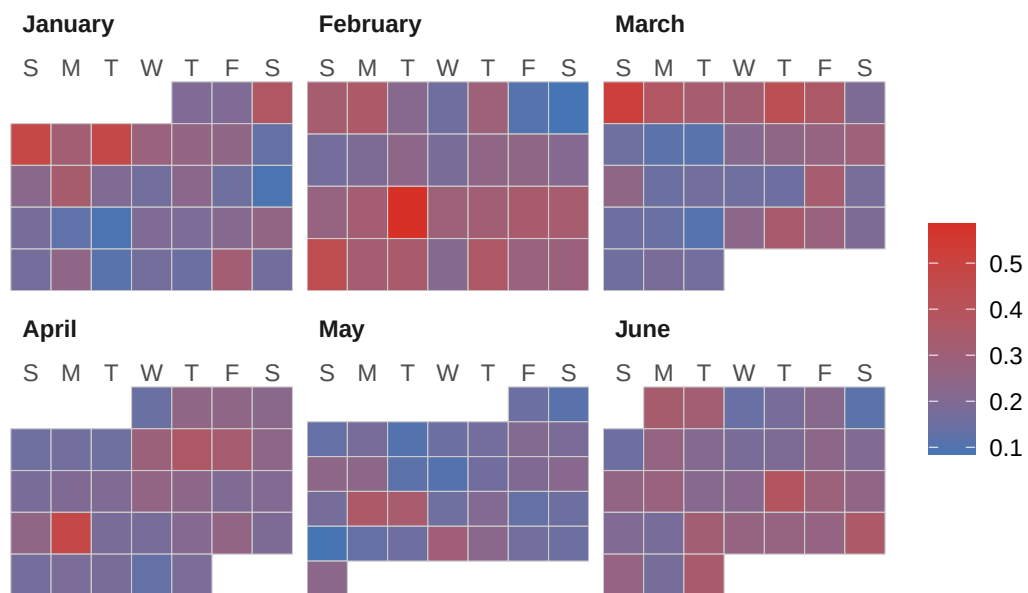
```
cUA + pal + ggtitle("United Airlines (UA)")
```

United Airlines (UA)



```
cUS + pal + ggtitle("US Airways (US)")
```

US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

[1] "2026-08-24 22:49:42 -03"